

Hierarchical Planning with JSHOP2

Aleksandra Słomska
Zofia Narloch

May 10, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

In this exercise we recreated the Emergency Service Logistics domain from the PDDL labs, but this time using JSHOP2, which is a hierarchical planner. The main difference is that instead of just describing what actions are possible (like in PDDL), we also have to tell the planner how to organise those actions into higher-level tasks.

Below is an example problem we defined for JSHOP2. There are three people, three locations and three crates:

```
1 (defproblem problem p1
2   (
3     (drone-at drone1 depot)
4     (crate-at crate1 depot)
5     (crate-at crate2 depot)
6     (crate-at crate3 depot)
7     (empty-arm drone1 left)
8     (empty-arm drone1 right)
9     (content-crate food crate1)
10    (content-crate food crate2)
11    (content-crate medicine crate3)
12    (person-at person1 loc1)
13    (person-at person2 loc3)
14    (person-at person3 loc2)
15    (needs-content person2 food)
16    (needs-content person2 medicine)
17    (needs-content person3 food)
18  )
19  (
20    (deliver-all)
21  )
22 )
```

Here the goal is just `(deliver-all)` — a single high-level task. In PDDL we would instead list each individual goal condition, like `(has-content-person person2 food)`. In JSHOP2 we let the task hierarchy figure out what needs to be done.

The planner produced the following plan:

```

1 (!pick-up-crate crate1 depot drone1 left)
2 (!drone-move depot loc3 drone1)
3 (!drop-crate loc3 person2 crate1 left food drone1)
4 (!drone-move loc3 depot drone1)
5 (!pick-up-crate crate3 depot drone1 right)
6 (!drone-move depot loc3 drone1)
7 (!drop-crate loc3 person2 crate3 right medicine drone1)
8 (!drone-move loc3 depot drone1)
9 (!pick-up-crate crate2 depot drone1 left)
10 (!drone-move depot loc2 drone1)
11 (!drop-crate loc2 person3 crate2 left food drone1)
12 -----
13 Time Used = 0.009

```

Listing 1: First problem and the outcome - three people and three boxes

The plan looks very similar to what PDDL planners produce — the individual actions are the same. The difference is in how the planner found the plan.

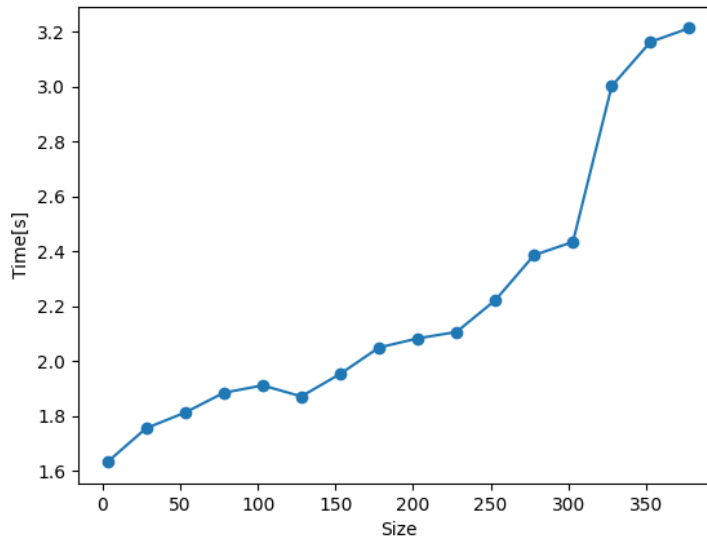


Figure 1: Problem size vs. planning time for Exercise 1.1

It is important to note that the time shown on the graph is measured by the Python script. The time reported by JSHOP2 itself was below 1 second for every single problem. The graph shows that even at size 380 the total time was around 3.2 seconds — and most of that is overhead from the Python script.

1.1 Comparison with Classical Planning (BFS from Exercise 1.3)

In Part 1.3 of the PDDL project, we tested several search algorithms. The best performing one was BFS (Breadth-First Search) with no heuristic. It solved a problem of size 17 in just 0.77 seconds. That was the largest problem any algorithm could solve within the 60-second time limit.

JSHOP2 solved a problem of size 380 in under one second of actual planning time. That is more than 22 times larger.

BFS works by expanding every possible state one step at a time, level by level. The number of states grows fast as the problem gets bigger: by size 17, BFS was already hitting the 60-second limit. JSHOP2 does not search through states in the same way — it follows a task hierarchy defined in the domain file. The top-level task (`deliver-all`) is broken down into smaller tasks, which are broken down further into concrete actions.

The downside is that JSHOP2 does not guarantee an optimal plan. In the example above, the drone goes to `loc3` twice (for food and medicine separately for `person2`), even though it could carry both crates at once. BFS would find the shortest possible plan.

1.2 Advantages of Hierarchical Planning

- **Scalability.** While BFS could not go past size 17, JSHOP2 handled size 380.
- **Speed.** The planner commits to a plan quickly by following the hierarchy; there is no exhaustive search.

1.3 Disadvantages of Hierarchical Planning

- **No optimality guarantee.** JSHOP2 made two separate trips to deliver to `person2` even though one trip would have sufficed.
- **More work upfront.** Writing the JSHOP2 domain requires designing the full task hierarchy and defining methods for every situation.

2 Exercise 1.2: Advanced Emergency Logistics Domain

In the second part of the exercise we upgraded our JSHOP2 domain to handle more realistic logistics using numerical fluents and carrier. The domain moves away from individual named crates and instead tracks aggregate quantities of goods at locations, making the representation much more compact and scalable.

2.1 Key Representational Changes

Unlike Part 1, where every crate was a distinct object, the advanced domain represents supply as numeric quantities:

- `(crates-at food depot 4)` – 4 food boxes are at the depot.
- `(needs food loc3 1)` – location `loc3` needs 1 food box.
- `(carrier-at carrier1 depot)` and `(carrier-capacity carrier1 5)` – a carrier with capacity 5 is parked at the depot.
- `(carrier-content carrier1 food 0)` – the carrier currently carries no food.
- `(reserved carrier1 food loc3 4)` – 4 food boxes are reserved for `loc3` in `carrier1`.

This numeric representation removes the need to enumerate individual crate objects and allows the planner to reason about partial loads and bulk deliveries using calls such as `(call >= ?count 1)`.

2.2 Delivery Strategy

The `deliver-all` method selects among four branches, tried in order of priority:

1. **Arm delivery for small needs (`use-drone-arms-small`).** When the greatest remaining need is at most 2 boxes, the drone uses its own arms (left/right) instead of a carrier. The `fill-arms` method loads one box per free arm, and `dispatch-arms` flies to each target location, drops the box, and returns to the depot.
2. **Carrier fits the need (`use-carrier-fits`).** A carrier whose capacity $\geq$ the need is selected. Among all fitting carriers, the smallest one is chosen via the `better-fitting-carrier-exists` axiom.
3. **No carrier fits (`use-carrier-partial`).** When no single carrier can hold the entire need, the largest available carrier is selected via the `larger-available-carrier-exists` axiom, and it is filled to capacity.
4. **Arm fallback (`use-drone-arms`).** When no carrier is available at all (or none was selected by the above branches), the drone delivers one box per arm.

For carrier-based branches, each dispatch cycle has two phases:

1. **Filling phase.** The drone stays at the depot and loads boxes into the chosen carrier via the `fill-specific` and `fill-any` methods. Loading continues until the carrier is full or all current needs are reserved.

2. **Dispatch phase.** The drone flies the carrier along a multi-stop route via the `dispatch-carrier` method, calling `!unload-carrier` at each reserved location before returning to the depot.

After each return to depot, `deliver-all` recurses and re-evaluates the remaining needs, selecting the next best branch.

2.3 Carrier Selection with Axioms

Carrier selection is driven by axioms that evaluate the current state at planning time:

- `better-fitting-carrier-exists ?need ?cap` — true if a carrier with capacity $\geq$ `need` but $<$ `cap` exists. Used to ensure the smallest fitting carrier is always chosen first.
- `carrier-can-fit ?need` — true if at least one carrier can fit the entire need in one load.
- `larger-available-carrier-exists ?cap` — true if a larger carrier than the current candidate exists. Used when no carrier fits the need, to ensure the largest available is chosen.
- `greater-need-exists ?count` — true if any location has a higher need than the current one being considered. This implements a greater priority.

The `deliver-all` method tries four branches in order:

```

1 use-drone-arms-small
2 (
3   (needs ?type ?loc ?need-count)
4   (call <= ?need-count 2)
5   (call >= ?need-count 1)
6   (not (greater-need-exists ?need-count))
7   (empty-arm drone1 ?arm)
8 )
9
10 use-carrier-fits
11 (
12   (needs ?type ?loc ?need-count)
13   (call >= ?need-count 1)
14   (not (greater-need-exists ?need-count))
15   (carrier-at ?carrier depot)
16   (carrier-capacity ?carrier ?cap)
17   (call >= ?cap ?need-count)
18   (not (better-fitting-carrier-exists ?need-count ?cap))
19 )
20
21 use-carrier-partial
22 (
23   (needs ?type ?loc ?need-count)
24   (call >= ?need-count 1)
25   (not (greater-need-exists ?need-count))

```

```

26 (not (carrier-can-fit ?need-count))
27 (carrier-at ?carrier depot)
28 (carrier-capacity ?carrier ?cap)
29 (not (larger-available-carrier-exists ?cap))
30 )
31
32 use-drone-arms
33 (
34   (needs ?type ?loc ?need-count)
35   (call >= ?need-count 1)
36   (empty-arm drone1 ?arm)
37 )

```

Listing 2: Delivery branches in deliver-all (p3.3)

2.4 Partial and Full Loading

The `fill-any` and `fill-specific` method handles two sub-cases:

- `load-fits` – the entire remaining need fits in the available capacity; all of it is reserved and loaded.
- `load-partial` – the need exceeds the remaining capacity; only some boxes are reserved and loaded, leaving the rest for a future cycle.

2.5 Arm-Based Delivery: fill-arms and dispatch-arms

Both the `use-drone-arms-small` and the fallback `use-drone-arms` branches use the same sub-methods:

- `fill-arms` recursively loads one box per free arm using the `!pick-up-crate-arm` operator. This operator simultaneously picks up a box from the depot and reserves the destination by decrementing the `needs` counter. Loading stops when either no empty arms remain or all needs are satisfied.
- `dispatch-arms` recursively visits each target location, drops the held box with `!drop-crate-arm`, and continues until the drone’s arms are empty. It then returns the drone to the depot.

```

1 use-drone-arms-small
2 (
3   ...
4 )
5 (
6   (move-drone-alone drone1 depot)
7   (fill-arms drone1)
8   (dispatch-arms drone1)
9   (deliver-all)
10 )

```

Listing 3: Arm-based delivery flow in deliver-all

The **use-drone-arms-small** branch is an optimisation: when the greatest remaining need is small (≤ 2), the drone’s arms are sufficient, and deploying a carrier would be higher cost. The fallback **use-drone-arms** branch handles any remaining needs when no carriers are available.

2.6 Flight Cost Model

The cost of moving a drone without a carrier has a fixed cost of 50. Moving a carrier adds an additional cost proportional to its capacity, that is why a heavier or larger vehicle is more expensive to operate:

$$\text{cost}(\text{carrier move}) = 50 + \frac{\text{capacity}}{10}$$

For example, a carrier with capacity 100 costs 60 per flight, while a capacity-5 carrier costs 50.5. This makes the planner to use smaller carrier when they are sufficient.

2.7 Handling the Required Situations

Table 1 summarises how each of the 11 required situations is handled by the domain.

#	Situation	Mechanism
1	No carrier available	use-drone-arms
2	One carrier available	use-carrier-fits , use-carrier-partial
3	Different box types on the same carrier	fill-specific , fill-any
4	Remaining boxes delivered without carrier after carrier cycle	deliver-all $\rightarrow$ use-drone-arms
5	carrier smaller than need $\rightarrow$ load to max capacity	load-partial in fill-specific/fill-any
6	carrier larger than need $\rightarrow$ load only what is needed	load-fits in fill-specific/fill-any
7	Multiple carriers fit $\rightarrow$ choose smallest fitting	better-fitting-carrier-exists axiom
8	No carrier fits $\rightarrow$ choose largest available	larger-available-carrier-exists axiom
9	Re-evaluate after each depot return	deliver-all , dispatch-carrier
10	Serve location with greatest need first	greater-need-exists
11	Multi-location trip without returning to depot	dispatch-carrier

Table 1: The 11 required situations and how they are handled in the domain.

2.8 Scalability Comparison

The P2 domain produces plans much faster than the P1 domain for problems with many deliveries, because needs are accumulated per location. Instead of making one trip per crate, the planner loads an entire carrier and serves multiple locations in one dispatch cycle, reducing the total number of operators in the plan and the depth of the search.